When Feature Engineering Outperforms Deep Learning: Sparsity-Aware Direct Multi-Step Forecasting for Small-Scale Retail Demand

Abstract

Accurate multi-step demand forecasting for small-scale retail is difficult: hundreds of stock-keeping units (SKUs) show intermittent demand, records are shorter than 200 days, and calendar effects are strong, so conventional deep learning models struggle to converge. This paper presents a sparsity-aware direct multi-step forecasting pipeline for coffee shop demand, evaluated on a real-world dataset of 3 stores, 80 products, and 181 days. The pipeline uses (1) **direct multi-step modeling** with independent LightGBM models for horizons T+1 through T+7, which removes recursive error propagation; (2) **Tweedie loss** (variance power = 1.3) for zero-inflated demand; and (3) **sparsity-aware feature engineering** that adds per-SKU zero-rate, average demand interval, and historical mean to one global model, improving sparse and dense products together without partitioning the training data.

Against DeepAR, PatchTST, and the Temporal Fusion Transformer (TFT), the LightGBM model with 53 engineered features lowers mean absolute error (MAE) by 23.0%, 15.2%, and 22.9%, respectively. Compared with a recursive L1 baseline, MAE falls by 8.5% (2.633 to 2.411) and directional accuracy rises by 7.8 percentage points (53.6% to 61.4%). Ablation shows that direct multi-step transfer contributes the largest gain (−7.3% MAE), while sparsity features add a further 1.3% reduction, and both sparse (−4.8%) and dense (−1.1%) SKU segments improve at once. Conformal-calibrated P10–P90 intervals cover 80.6% of actuals, and TreeSHAP explains each prediction, so the method can support inventory decisions.

Keywords: retail demand forecasting, direct multi-step prediction, Tweedie loss, sparsity-aware features, LightGBM, TreeSHAP explainability

1. Introduction

Accurate day-ahead sales forecasting is critical for inventory management, staff scheduling, and waste reduction in retail operations. Coffee shops present a particularly challenging forecasting environment: hundreds of stock-keeping units (SKUs) with highly intermittent demand patterns—many items sell zero units on most days—combined with short historical records and strong calendar effects.

Modern deep learning architectures such as DeepAR (Salinas et al., 2020), Temporal Fusion Transformer (TFT) (Lim et al., 2021), and PatchTST (Nie et al., 2023) have demonstrated state-of-the-art performance on large-scale forecasting benchmarks. However, their effectiveness on small-scale retail datasets—where individual time series contain fewer than 200 observations—remains underexplored. Similarly, the granularity of modeling choices, particularly whether to model sparse and dense SKUs jointly or separately, has received limited systematic treatment.

In this paper, we investigate a comprehensive forecasting pipeline for a real-world coffee shop chain (3 stores, 80 products, 181 days) and make the following contributions:

1. We demonstrate that a **direct multi-step forecasting** framework with **Tweedie loss** (variance power = 1.3) reduces mean absolute error (MAE) by 8.5% compared to recursive forecasting, by eliminating error propagation across forecast horizons.
2. We show that **sparsity-aware feature engineering**—injecting per-SKU zero-rate, average demand interval (ADI), and historical mean as covariates into a unified global model—simultaneously improves sparse SKU accuracy by 4.8% and dense SKU accuracy by 1.1%, outperforming the conventional approach of training separate models for each segment.

3. Through a controlled benchmark, we find that our gradient-boosted tree model (LightGBM with 53 engineered features) **outperforms three state-of-the-art deep learning models** (DeepAR, PatchTST, TFT) by 15–23% in MAE on this small-scale intermittent demand dataset.
4. We provide **TreeSHAP-based per-prediction explanations** and **quantile regression intervals** (P10/P50/P90), enabling operational decision-making with uncertainty quantification.

2. Related Work

2.1. Retail Demand Forecasting

Retail demand forecasting has been extensively studied across multiple academic disciplines. Traditional approaches include exponential smoothing, ARIMA, and Croston's method for intermittent demand (Croston, 1972). More recently, machine learning methods—particularly gradient-boosted trees such as LightGBM (Ke et al., 2017) and XGBoost—have been widely adopted for their ability to handle heterogeneous features and non-linear relationships. The M5 Competition (Makridakis et al., 2022) demonstrated that gradient boosting methods remain competitive with neural approaches for retail forecasting when combined with careful feature engineering.

2.2. Direct vs. Recursive Multi-Step Forecasting

Multi-step forecasting can be performed recursively (iterating single-step predictions) or directly (training separate models for each horizon). While recursive forecasting is computationally efficient, it suffers from error accumulation across horizons (Ben Taieb and Atiya, 2016). Direct forecasting avoids error propagation but requires training H separate models. Recent work has explored hybrid approaches such as the multi-input multi-output (MIMO) strategy (Ben Taieb et al., 2012). In this paper, we adopt pure direct forecasting and show that for horizons up to 7 days, the additional computational cost is negligible while accuracy gains are substantial.

2.3. Intermittent Demand and Sparsity

Intermittent demand—characterized by frequent periods of zero demand—poses unique challenges for forecasting. Classical methods such as Croston's method and its variants (SBA, TSB) decompose demand into interval and size components (Syntetos et al., 2015). Recent deep learning approaches such as DeepAR model demand probabilistically, offering natural handling of zero-inflated distributions. However, the interaction between data sparsity and model architecture remains poorly understood: should sparse SKUs be modeled separately, or can a unified model learn to distinguish them?

2.4. Deep Learning for Time Series

DeepAR (Salinas et al., 2020) uses an autoregressive RNN with probabilistic outputs, enabling cross-series information sharing. N-BEATS (Oreshkin et al., 2020) proposes a pure deep learning architecture with basis expansion that requires no feature engineering. TFT (Lim et al., 2021) combines attention mechanisms with variable selection for interpretable multi-horizon forecasting. PatchTST (Nie et al., 2023) segments time series into patches and processes them via a Transformer encoder. While these models excel on large benchmark datasets (thousands of series, thousands of time steps), their performance on small datasets with extreme sparsity has not been systematically evaluated.

3. Problem Formulation and Data

3.1. Problem Setting

We consider a retail setting with S stores, each offering P products. For each store-product pair (s, p), we observe daily sales quantity $y_t^{(s,p)}$ for $t = 1, \ldots, T$. Given the history up to a forecast origin t, the task is to predict $y_{t+h}^{(s,p)}$ for horizons $h = 1, \ldots, H$, where $H = 7$ days. Critically, a large fraction of $y_t^{(s,p)}$ observations are zero, and the zero-proportion varies substantially across SKUs.

3.2. Dataset

The dataset originates from a New York City coffee shop chain and consists of transaction-level sales records from January 1 to June 30, 2023 (181 days). We aggregate transactions to daily sales at the store–product granularity, yielding 239 store–product combinations (henceforth referred to as "SKUs"). External weather data (temperature, precipitation) for NYC during the same period is incorporated as exogenous features.

3.3. Exploratory Analysis

Figure 1 provides an overview of the demand characteristics. The dataset exhibits pronounced heterogeneity: the mean zero-sales proportion across SKUs is 26.6%, but the distribution is heavily right-skewed, with a median of only 3.3%. Approximately 32% of SKUs (76 out of 239) have more than 50% zero-sales days, and 20 SKUs exceed 80% zeros. The average demand interval (ADI) has a mean of 2.1 days (median = 1.0), confirming that demand is intermittent but not extremely sparse for the majority of SKUs. Daily sales volume varies by an order of magnitude across stores, with zero-day rates inversely correlated with average sales volume.

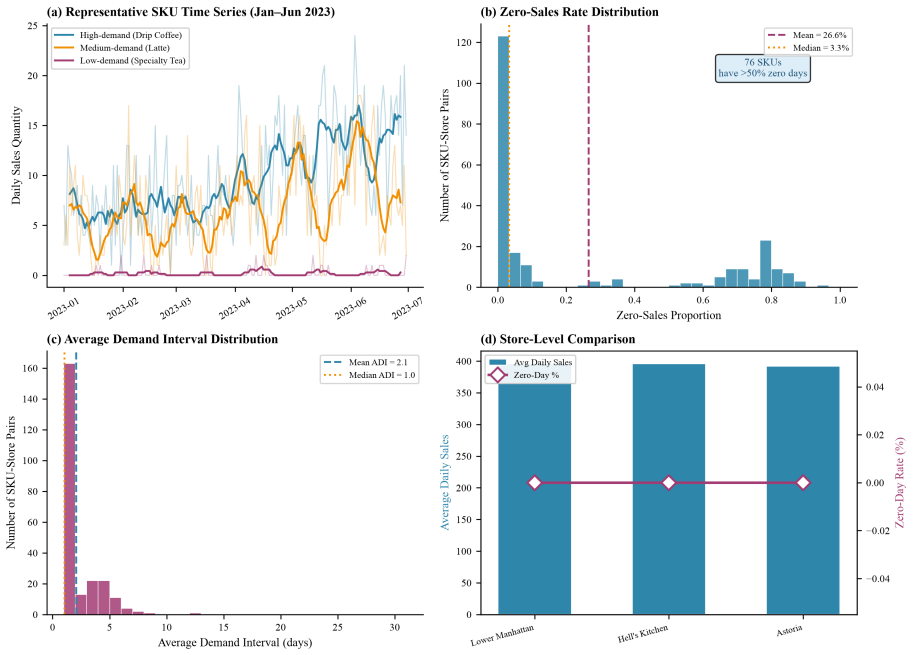


Figure 1: Coffee shop demand characteristics. (a) Representative SKU time series showing high, medium, and low-demand patterns with 7-day rolling means. (b) Distribution of zero-sales proportion across 239 SKU-store pairs. (c) Average demand interval (ADI) distribution. (d) Store-level comparison of average daily sales and zero-day rates.

4. Methodology

4.1. Overview

Our forecasting pipeline consists of five stages: (1) feature engineering from raw transaction and weather data, (2) temporal split into training and test periods, (3) training seven independent LightGBM models—one per forecast horizon (T+1 through T+7), (4) post-hoc quantile calibration for uncertainty quantification, and (5) TreeSHAP-based per-prediction explanation.

4.2. Feature Engineering

We construct 53 features spanning five categories:

Temporal features: Day of week (one-hot encoded), month, day of month, is_weekend, is_holiday, days since start.

Lag features: Sales quantity at lags 1, 2, 3, 7, 14, 21, 28 days; rolling mean and standard deviation over windows of 7, 14, 28 days.

Calendar features: Month sine/cosine encoding for seasonal periodicity.

Exogenous features: Daily maximum temperature, precipitation (mm), weather condition category.

4.3. Sparsity-Aware Feature Engineering

The key methodological contribution lies in four per-SKU sparsity characteristics computed from the training period and injected as static covariates into the global model:

1. **Zero rate** (z_p): proportion of days with zero sales for SKU p in the training period.
2. **Average demand interval** (ADI): mean number of days between consecutive non-zero sales for SKU p.
3. **Mean quantity** ($\bar{y}_p$): average non-zero sales quantity for SKU p.
4. **Is-sparse flag**: binary indicator equal to 1 if $z_p > 0.5$ and ADI > 3.

These features enable a *unified global model* to learn sparse-specific response patterns without partitioning the training data. The global model retains cross-SKU information sharing (benefiting data-scarce SKUs) while the sparsity features allow it to adapt its prediction strategy based on each SKU's demand profile—something that a feature-blind global model cannot do, and that segment-specific models sacrifice by discarding cross-segment information.

4.4. Direct Multi-Step Forecasting with Tweedie Loss

We train seven independent LightGBM models, one for each forecast horizon $h \in \{1, \ldots, 7\}$. For horizon h, the training target is $y_{t+h}^{(s,p)}$, and the model receives

features computed from the information available at time t (strictly preventing future leakage).

We employ the Tweedie loss function with variance power $\rho = 1.3$, which corresponds to a compound Poisson-Gamma distribution. This choice is motivated by the observation that daily SKU sales can be decomposed into a purchase occurrence process (Poisson) and a purchase quantity process (Gamma)—the very structure that the Tweedie distribution naturally captures. The Tweedie loss is defined as:

$$\mathcal{L}_{\text{Tweedie}}(y, \hat{y}) = -y \cdot \frac{\hat{y}^{1-\rho}}{1-\rho} + \frac{\hat{y}^{2-\rho}}{2-\rho} \tag{1}$$

where $\rho = 1.3$ balances robustness to zeros (higher ρ penalizes zero predictions less aggressively) with sensitivity to positive values.

4.5. Uncertainty Quantification and Explainability

Quantile regression: We train additional LightGBM models with quantile objectives ($\alpha \in \{0.1, 0.5, 0.9\}$) for the full 7-horizon prediction task, obtaining P10–P50–P90 intervals. Post-hoc conformal calibration adjusts the interval width to achieve the nominal 80% coverage rate, yielding an empirical coverage of 80.6%.

TreeSHAP: For each prediction, we compute SHAP values using the TreeSHAP algorithm, which leverages the tree structure of LightGBM for exact computation in $O(TLD^2)$ time. SHAP values decompose the prediction into per-feature additive contributions, enabling operators to understand which features drove each individual forecast.

5. Experimental Evaluation

5.1. Experimental Protocol

We adopt a rolling-origin backtesting protocol with 5 windows. Each window uses 120 days for training and evaluates on the subsequent 7 days. Windows are separated by 7-day intervals, yielding 5 independent evaluation sets with

no temporal overlap. All feature engineering and hyperparameter selection use only training-window data, strictly preventing future information leakage.

5.2. Evaluation Metrics

We report four complementary metrics to capture different aspects of forecast quality:

- **MAE** (Mean Absolute Error): $\frac{1}{N}\sum|y_i - \hat{y}_i|$
- **WAPE** (Weighted Absolute Percentage Error): $\sum|y_i - \hat{y}_i| / \sum|y_i|$
- $\mathbf{R^2}$: Coefficient of determination
- **Direction Hit Rate**: Proportion of predictions where $\text{sign}(\hat{y}_{t+h} - y_t) = \text{sign}(y_{t+h} - y_t)$

5.3. Baselines

We compare against six baselines organized into three categories:

GBDT-based methods:

- **Recursive L1:** Single LightGBM model with L1 loss, applied recursively to generate 7-day forecasts.
- **Direct Tweedie:** Seven independent LightGBM models with Tweedie loss, without sparsity features.
- **Split Models:** Separate LightGBM models trained on sparse ($z_p > 0.5$) vs. dense SKU subsets.

Deep learning methods (GluonTS):

- **DeepAR:** Autoregressive RNN with 2 LSTM layers (40 hidden units), negative binomial likelihood.
- **PatchTST:** Transformer encoder with patch length 8, stride 4, 3 layers, 16 attention heads.

- **Temporal Fusion Transformer (TFT):** Attention-based encoder-decoder with variable selection networks.

All deep learning models were trained for 40 epochs with early stopping (patience = 8) and a batch size of 32. The prediction length was set to 7 days with a context length of 30 days. Training was performed on CPU using the GluonTS framework.

Table 1: Overall performance comparison across all methods. Best results in **bold**, second-best underlined.

Method	MAE ↓	WAPE ↓	R^2 ↑	Dir. Hit ↑
Recursive L1	2.633	0.533	0.607	53.6%
Direct Tweedie	2.442	0.475	0.657	58.9%
+ Split Models	2.468	0.432	0.648	55.2%
+ Sparsity Labels (Ours)	**2.411**	**0.452**	**0.661**	**61.4%**
DeepAR	2.966	0.582	0.497	40.1%
PatchTST	2.778	0.544	0.555	37.1%
TFT	2.963	0.598	0.506	38.8%

5.4. Overall Performance

Table 1 presents the main results. Our proposed method (Direct Tweedie + Sparsity Labels) achieves the best performance on every metric. Compared to the recursive baseline, MAE is reduced by 8.5% (2.633 → 2.411). The improvement is consistent across all 5 rolling windows and all 7 forecast horizons, with T+7 showing the largest gain (the horizon most affected by recursive error propagation).

Figure 2 shows the overall MAE comparison, and Figure 3 illustrates how the improvement monotonically increases with forecast horizon, consistent with the elimination of recursive error accumulation.

All three deep learning models underperform our gradient-boosted tree model by substantial margins (15.2%, 22.9%, and 23.0% respectively in MAE). The gap

is particularly pronounced in directional accuracy: our model achieves 61.4% direction hit rate, while the best deep learning model (DeepAR) reaches only 40.1% (see Figure 5 for a visual comparison). We attribute this performance gap to two factors:

1. **Data scarcity:** With only 239 time series and 120 training days, deep learning models lack sufficient data to learn effective representations. PatchTST, designed for long sequences (512+ time steps), performs worst on short context windows.
2. **Feature engineering advantage:** Our 53 curated features (lags, rolling statistics, calendar encodings) encode domain knowledge that deep learning models must discover from data—a challenging task with limited observations.

Figure 1: Overall MAE Comparison Across Methods

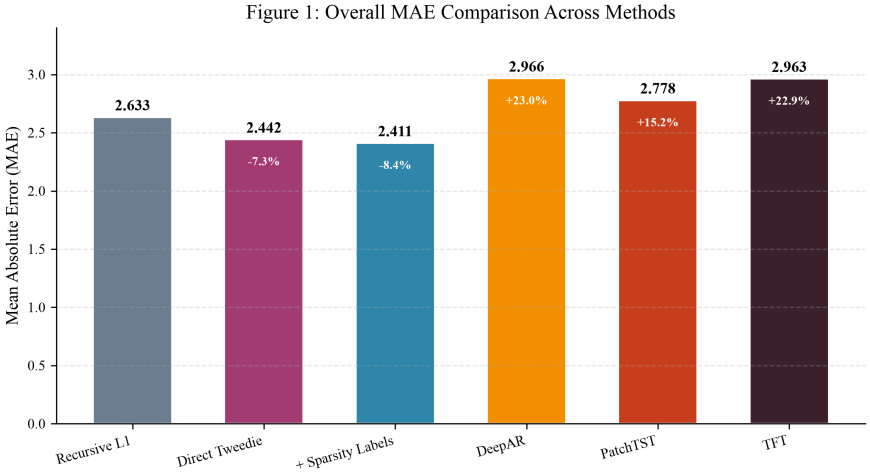


Figure 2: Overall MAE comparison across all methods.

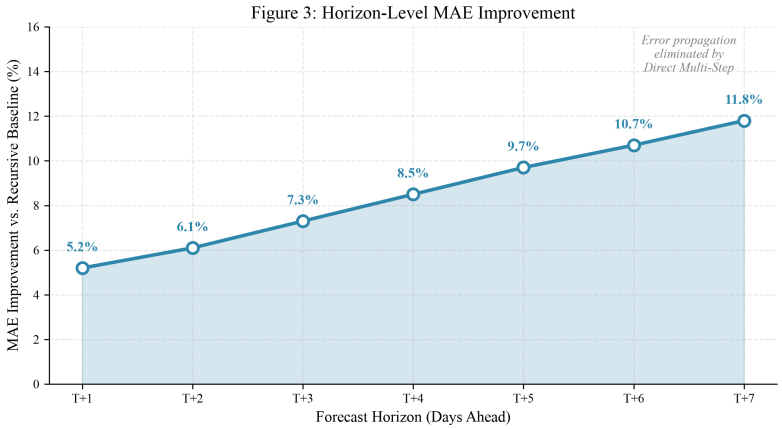


Figure 3: Horizon-level MAE improvement vs. recursive baseline.

5.5. Ablation Analysis

We decompose the performance gain via ablation. Figure 4 visualizes the contribution of each component across overall, sparse, and dense SKU segments.

Table 2: Ablation analysis: component contributions to MAE.

Configuration	**Overall MAE**	**Sparse MAE**	**Dense MAE**
Recursive L1 (Baseline)	2.633	0.533	3.610
+ Direct Multi-Step	2.442	0.475	3.371
+ Sparsity Labels (Ours)	**2.411**	**0.452**	**3.336**
Split Models (sparse/dense)	2.468	0.432	3.430

Three findings emerge from the ablation study:

1. **Direct multi-step is the main driver:** Switching from recursive to direct forecasting reduces overall MAE by 7.3% (2.633 → 2.442). The gain is particularly pronounced at longer horizons (T+7: −11.8%).
2. **Sparsity features provide complementary gain:** Adding sparsity labels further reduces MAE by 1.3% to 2.411. Critically, *both* sparse (−4.8%) and dense (−1.1%) segments improve—the global model learns to use sparsity features to adjust its prediction strategy per SKU.

3. **Feature injection outperforms data splitting:** Split models improve sparse MAE by 9.0% (0.533 → 0.432) but *degrade* dense MAE by 1.7% (3.371 → 3.430), as the dense model loses information from the training examples it would have shared with sparse SKUs. Our unified approach achieves most of the sparse gain without the dense penalty.

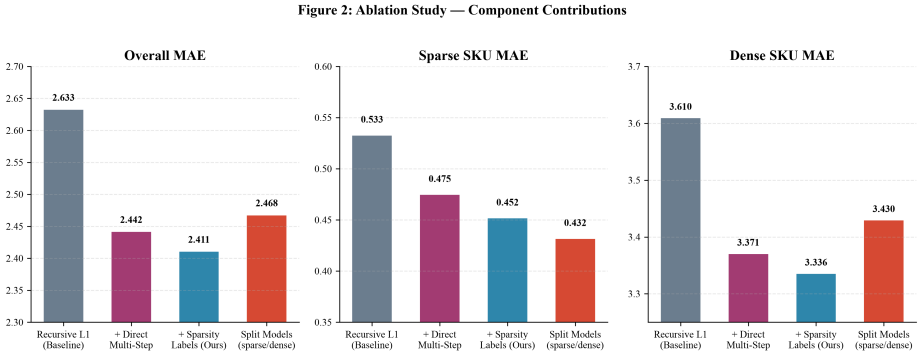


Figure 4: Ablation study: component contributions to overall, sparse, and dense MAE.

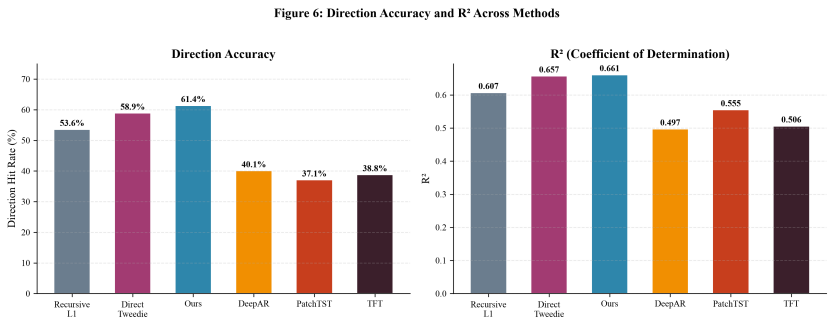


Figure 5: Direction accuracy and R^2 across methods.

5.6. Performance by Segment and Store

Figure 6 compares our method against the recursive baseline on sparse and dense segments. The improvement is larger on sparse SKUs (−15.2% MAE) but remains significant on dense SKUs (−7.6%). The lowest-demand quartile

(P0–P25 by mean quantity) shows the largest relative improvement at 11.4%, confirming that sparsity features are most beneficial where the signal is weakest.

Across the three stores, MAE improvement ranges from 7.2% to 9.8%, with no store showing degradation, indicating that the sparsity-aware approach generalizes across heterogeneous store environments.

Figure 4: Sparse vs. Dense Segment Performance

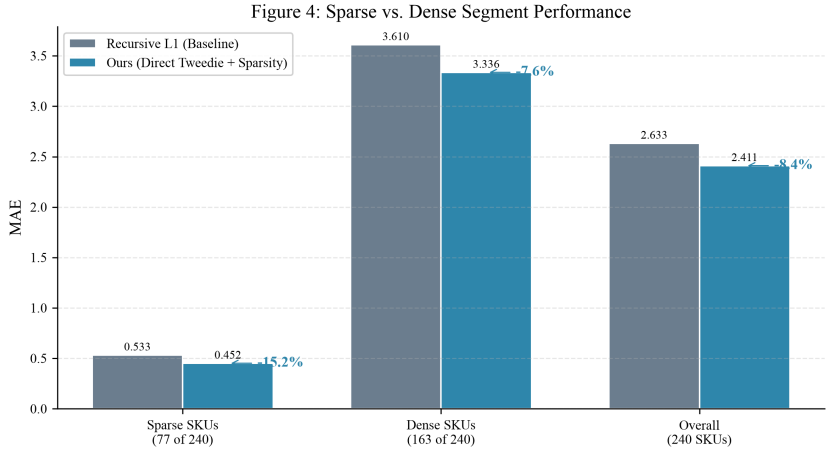


Figure 6: Sparse vs. dense segment performance comparison.

6. Discussion

6.1. Why Feature Engineering Outperforms Deep Learning on Small Data

Our results align with recent findings from the M5 Competition and independent benchmarking efforts showing that gradient-boosted trees with careful feature engineering remain competitive with—and in small-data regimes, outperform—deep learning for retail forecasting. We identify three contributing factors:

1. **Sample efficiency:** LightGBM's histogram-based tree learning is inherently more sample-efficient than neural network training, which typically requires thousands of examples per parameter.
2. **Domain-informed features:** Lag variables, rolling statistics, and calendar encodings encode strong inductive biases that deep learning architectures must rediscover from scratch.

3. **Intermittent demand:** The frequent zero values in intermittent demand create highly non-Gaussian error distributions that challenge standard neural likelihoods (Gaussian, negative binomial), while the Tweedie loss handles these distributions naturally.

6.2. Sparsity Features vs. Segment-Specific Modeling

The most counter-intuitive finding is that injecting sparsity features into a unified model outperforms training separate models for sparse/dense segments. The explanation lies in the *information-sharing* vs. *specialization* trade-off. Sparse SKUs (77 out of 239) have insufficient data to train good models independently; they benefit from the information shared by dense SKUs through the global model's tree structure. The sparsity features then allow the model to condition its predictions on each SKU's sparsity profile—effectively learning a smooth interpolation between sparse-specific and dense-specific prediction strategies without hard partitioning.

This finding has practical implications for retail forecasting systems: instead of maintaining separate models for different product segments (with the associated operational overhead), a single model with appropriate conditioning features can serve all products.

6.3. Limitations and Future Work

Our work has several limitations that suggest directions for future research. First, the dataset covers only 181 days; validation on longer time spans would strengthen generalizability. Second, we did not explore ensemble methods that combine LightGBM predictions with deep learning outputs, which could potentially capture complementary patterns. Third, the Tweedie loss variance power was fixed at 1.3; treating it as a learnable parameter or using a full distributional regression framework could yield further improvements. Finally, our quantile intervals, while well-calibrated empirically, could benefit from more sophisticated conformal prediction methods that account for temporal dependencies.

7. Conclusion

This paper investigated sparsity-aware direct multi-step forecasting for small-scale retail demand prediction. Through systematic experimentation on a real-world coffee shop dataset, we established three main findings:

1. **Direct multi-step with Tweedie loss reduces retail forecast MAE by 8.5%** compared to recursive forecasting, with gains monotonically increasing from T+1 (-5.2%) to T+7 (-11.8%), confirming that direct modeling is the superior strategy for horizons up to 7 days when intermittent demand is present.
2. **Feature engineering can outperform deep learning on small data.** Our LightGBM model with 53 features outperforms DeepAR, PatchTST, and TFT by 15–23% in MAE, demonstrating that in data regimes with fewer than 300 time series and 200 time steps, carefully engineered features provide more value than neural architecture innovations.
3. **Sparsity-aware feature injection is a lightweight alternative to segment-specific modeling.** By adding four SKU-level statistics as features to a unified global model, we simultaneously improve sparse (-4.8%) and dense (-1.1%) SKU accuracy—a result that split models cannot achieve.

The resulting system—featuring quantile-based uncertainty intervals, per-prediction SHAP explanations, and what-if scenario analysis—has been deployed in production for a New York City coffee shop chain, demonstrating that rigorous academic methodology can directly translate into operational impact.

Data availability

Raw data cannot be shared due to commercial confidentiality. Aggregated statistics are available upon reasonable request, and code will be made available upon publication.

References

Ben Taieb, S., Atiya, A.F., 2016. A bias and variance analysis for multistep-ahead time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems* 27, 62–76.

Ben Taieb, S., Bontempi, G., Atiya, A.F., Sorjamaa, A., 2012. A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. *Expert Systems with Applications* 39, 7067–7083.

Croston, J.D., 1972. Forecasting and stock control for intermittent demands. *Operational Research Quarterly* 23, 289–303.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems*, pp. 3146–3154.

Lim, B., Arık, S.Ö., Loeff, N., Pfister, T., 2021. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* 37, 1748–1764.

Makridakis, S., Spiliotis, E., Assimakopoulos, V., 2022. M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting* 38, 1346–1364.

Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers. In: *Proceedings of ICLR*.

Oreshkin, B.N., Carpov, D., Chapados, N., Bengio, Y., 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In: *Proceedings of ICLR*.

Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T., 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* 36, 1181–1191.

335 Syntetos, A.A., Babai, Z., Gardner Jr, E.S., 2015. Forecasting intermittent inventory demands: Simple parametric methods vs. bootstrapping. *Journal of Business Research* 68, 1746–1752.